from the `head` node, and always locking the next mutex before releasing the current one, so there's no possibility of different lock orders in different threads. The only potential candidate for a race condition is the deletion of the removed node in `remove_if()` ⓴ because you do this after you've unlocked the mutex (it's undefined behavior to destroy a locked mutex). However, a few moments' thought reveals that this is indeed safe, because you still hold the mutex on the previous node (`current`), so no new thread can try to acquire the lock on the node you're deleting.

What about opportunities for concurrency? The whole point of such fine-grained locking was to improve the possibilities for concurrency over a single mutex, so have you achieved that? Yes, you have: different threads can be working on different nodes in the list at the same time, whether they're just processing each item with `for_each()`, searching with `find_first_if()`, or removing items with `remove_if()`. But because the mutex for each node must be locked in turn, the threads can't pass each other. If one thread is spending a long time processing a particular node, other threads will have to wait when they reach that particular node.

6.4. Summary

This chapter started by looking at what it means to design a data structure for concurrency and providing some guidelines for doing so. We then worked through several common data structures (stack, queue, hash map, and linked list), looking at how to apply those guidelines to implement them in a way designed for concurrent access, using locks to protect the data and prevent data races. You should now be able to look at the design of your own data structures to see where the opportunities for concurrency lie and where there's potential for race conditions.

In chapter 7 we'll look at ways of avoiding locks entirely, using the low-level atomic operations to provide the necessary ordering constraints, while sticking to the same set of guidelines.